

P12 - Informe de Control y Análisis

Estabilidad del modelo predictivo frente al Undersampling

David Torres - Victor Malsam

11/09/2026

- 1 Para qué sirve este documento
- 2 1. Pregunta del experimento
- 3 2. De dónde sale realmente cada dato
 - 3.1 2.1 Qué parte del notebook define el escenario y la semilla
 - 3.2 2.2 Qué parte del notebook aplica el undersampling
 - 3.3 2.3 Qué parte del notebook cuenta los registros antes y después
 - 3.4 2.4 Qué parte del notebook calcula AUC y busca los hiperparámetros
 - 3.5 2.5 Qué parte del notebook genera los 11 cortes para Kaggle
 - 3.6 2.6 Qué parte del notebook guarda PARAM.yml
- 4 2.7 Qué hace cada script Python
 - 4.1 Paso 1 - p12_01_descargar_submissions.py
 - 4.2 Paso 2 - p12_02_consolidar_kaggle.py
 - 4.3 Paso 3 - p12_03_analizar_cortes.py
 - 4.4 Paso 4 - p12_04_integrar_resultados.py
- 5 2.8 Entonces, ¿quién genera los archivos de 00_GLOBAL ?
- 6 3. Estado automático de las corridas
- 7 4. Construcción automática de la tabla central
- 8 5. Resumen por escenario
- 9 6. Comparación de cada escenario contra su propio Baseline
- 10 7. Ganancia máxima
- 11 8. AUC
- 12 9. Registros utilizados
- 13 10. Tiempo de ejecución
- 14 11. Relación tiempo - ganancia
- 15 12. Hiperparámetros ganadores y niter
- 16 13. Cortes de Kaggle
- 17 14. Comparación de escenarios todos contra todos
 - 17.1 14.1 Construcción automática de las diez comparaciones
 - 17.2 14.2 Resumen descriptivo de las diez comparaciones
 - 17.3 14.3 Wilcoxon pareado preliminar - todos contra todos
 - 17.4 14.4 Tabla de Wilcoxon preliminar
 - 17.5 14.5 Qué archivos nuevos deja esta sección
- 18 15. ¿Estamos listos para la comparación estadística?
- 19 16. Qué tenemos hoy y qué falta
- 20 17. Archivos automáticos que deja este informe
- 21 18. Mapa mental del P12
 - 21.1 Idea fundamental
 - 21.2 Lectura provisional actual

1 Para qué sirve este documento

Este es **nuestro informe interno de control y comprensión del P12**. No es la presentación de YouTube ni pretende volver a ejecutar el experimento.

El trabajo pesado sigue ocurriendo donde corresponde:

Jupyter + LightGBM + Grid Search + Kaggle

|

v

scripts Python

|

v

90_RESULTADOS

|

v

ESTE INFORME R MARKDOWN

|

v

tablas + gráficos + control

La idea es simple: **cada vez que aparezcan nuevos resultados consolidados en 90_RESULTADOS , hacemos Knit nuevamente y el informe se actualiza solo.**

El documento no vuelve a calcular el score de Kaggle ni modifica notebooks, modelos, parámetros o submissions.

2 1. Pregunta del experimento

El Problema #12 estudia:

¿Hasta cuánto podemos reducir los registros utilizados mediante undersampling sin deteriorar de manera relevante el comportamiento predictivo y económico del modelo?

Los cinco escenarios son:

Código	Escenario	Interpretación
B100	100% Baseline	
U040	40% Conserva aproximadamente 40% de CONTINUA	
U010	10% Conserva aproximadamente 10% de CONTINUA	
U005	5% Conserva aproximadamente 5% de CONTINUA	
U001	1% Conserva aproximadamente 1% de CONTINUA	

BAJA+1 y BAJA+2 se conservan completas. El porcentaje de undersampling se aplica sobre la clase CONTINUA; por eso el porcentaje real total conservado queda algo por encima del porcentaje nominal.

Las distintas semillas son **réplicas del mismo experimento**. Víctor y David no son dos experimentos distintos: las semillas de ambos se integrarán en un único análisis.

3 2. De dónde sale realmente cada dato

Esta sección es importante porque queremos poder volver dentro de varios días y saber **qué produjo cada número y qué archivo lo contiene.**

La idea central es:

El notebook .ipynb NO genera directamente la tabla maestra ni los gráficos globales.

El notebook ejecuta el experimento de una combinación:

SEMILLA + ESCENARIO

y deja dos fuentes fundamentales:

1. PARAM.yml
2. archivos de submission para Kaggle

Después, los scripts Python consultan Kaggle, consolidan los resultados y construyen los CSV de 90_RESULTADOS .

Finalmente, este R Markdown toma esos CSV ya consolidados y genera nuestro informe global en 91_ANALISIS/00_GLOBAL .

El recorrido completo es:

```
NOTEBOOK .ipynb
|
|-- PARAM.yml
|
|-- kaggle/KAP12_..._800.csv
|-- kaggle/KAP12_..._850.csv
|-- ...
|-- kaggle/KAP12_..._1300.csv
|
v
KAGGLE calcula Public Score
|
v
SCRIPTS PYTHON p12_01 ... p12_04
|
v
90_RESULTADOS/
|
v
P12_informe_control.Rmd
|
v
91_ANALISIS/00_GLOBAL/
```

3.1 2.1 Qué parte del notebook define el escenario y la semilla

En la celda de configuración del notebook se define la réplica que estamos ejecutando.

Ejemplo:

```

P12_SEMILLA <- 198427

P12_ESCENARIO <- "BASELINE_100"
P12_TRAINING_PCT <- 1.00
P12_CODIGO_ESCENARIO <- "B100"

PARAM$semilla_primigenia <- P12_SEMILLA
PARAM$experimento <- paste0(
  "P12_",
  P12_CODIGO_ESCENARIO,
  "_S",
  P12_SEMILLA
)

```

Para el P12, lo que cambia entre escenarios es principalmente:

```
P12_TRAINING_PCT
```

con los valores:

```

1.00
0.40
0.10
0.05
0.01

```

La semilla cambia entre réplicas.

3.2 2.2 Qué parte del notebook aplica el undersampling

En la sección **Training Strategy** el notebook lleva el porcentaje del escenario a:

```
PARAM$trainingstrategy$training_pct <- P12_TRAINING_PCT
```

Luego se construye `fold_train`.

El bloque fundamental es:

```

set.seed(
  PARAM$semilla_primigenia,
  kind = "L'Ecuyer-CMRG"
)

dataset[, azar := runif(nrow(dataset))]

dataset[
  ,
  fold_train :=
    foto_mes %in% PARAM$trainingstrategy$training &
    (
      clase_ternaria %in% c("BAJA+1", "BAJA+2") |
      azar < PARAM$trainingstrategy$training_pct
    )
]

```

Este es el punto concreto donde se produce el undersampling.

Interpretación:

```
BAJA+1 y BAJA+2 -> se conservan  
CONTINUA        -> queda según training_pct
```

Por eso un escenario 10% significa aproximadamente **10% de CONTINUA**, no 10% de todo el dataset.

3.3 2.3 Qué parte del notebook cuenta los registros antes y después

Inmediatamente después de crear `fold_train`, nuestras versiones finales del notebook guardan el efecto real del undersampling dentro de:

```
PARAM$p12$registros
```

El bloque contiene, entre otros:

```
PARAM$p12$registros$training_antes <- dataset[  
  foto_mes %in% PARAM$trainingstrategy$training,  
  .N  
]  
  
PARAM$p12$registros$training_despues <- dataset[  
  fold_train == TRUE,  
  .N  
]  
  
PARAM$p12$registros$porcentaje_real_conservado <- round(  
  100 *  
  PARAM$p12$registros$training_despues /  
  PARAM$p12$registros$training_antes,  
  4  
)
```

También quedan registrados:

```
positivos_antes  
positivos_despues  
negativos_antes  
negativos_despues
```

Estos datos después llegan al `PARAM.yml`.

3.4 2.4 Qué parte del notebook calcula AUC y busca los hiperparámetros

El notebook define primero los parámetros fijos de LightGBM.

Entre ellos:

```
max_bin = 31
learning_rate = 0.03
feature_fraction = 0.5
num_iterations = 2048
early_stopping_rounds = 200
```

Luego la función:

```
Estimar_AUC_lightgbm <- function(x)
```

entrena cada combinación del Grid Search y recupera:

```
AUC <- modelo_train$record_evals$valid$auc$eval[
  [modelo_train$best_iter]
]

niter <- modelo_train$best_iter
```

Es decir:

```
AUC    -> métrica de validación del modelo
niter  -> mejor iteración de boosting
```

Después el notebook evalúa las combinaciones del Grid y guarda:

```
tb_nueva[, c("AUC", "num_iterations")] :=
  Estimar_AUC_lightgbm(.SD),
  by = 1:nrow(tb_nueva)
]
```

La tabla del Grid también se escribe en:

```
tb_grid_search_01.txt
```

Finalmente se ordena por AUC y se conserva el ganador:

```
setorder(tb_nueva, -AUC)

PARAM$out$lgbm$AUC <- tb_nueva[1, AUC]

PARAM$out$lgbm$mejores_hiperparametros <-
  as.list(tb_nueva[1])
```

De ahí salen:

```
AUC
num_leaves
min_data_in_leaf
num_iterations / niter
```

que más tarde recupera el script `p12_04_integrar_resultados.py` .

3.5 2.5 Qué parte del notebook genera los 11 cortes

para Kaggle

En la sección:

```
Kaggle Competition Submit
```

el notebook define:

```
PARAM$kaggle$cortes <- seq(  
  800,  
  1300,  
  by = 50  
)
```

Por lo tanto se prueban:

```
800  
850  
900  
950  
1000  
1050  
1100  
1150  
1200  
1250  
1300
```

Para cada corte se crea un archivo:

```
archivo_kaggle <- paste0(  
  "./kaggle/KA",  
  PARAM$experimento,  
  "_",  
  envios,  
  ".csv"  
)
```

y luego se ejecuta:

```
kaggle competitions submit
```

Por ejemplo:

```
KAP12_B100_S198427_800.csv  
KAP12_B100_S198427_850.csv  
...  
KAP12_B100_S198427_1300.csv
```

Importante: el notebook genera y envía esos archivos, pero la **ganancia / Public Score** no la calcula el notebook.

La ganancia la calcula Kaggle después de recibir cada submission.

3.6 2.6 Qué parte del notebook guarda PARAM.yml

Al final de cada corrida se calcula el tiempo:

```
P12_FIN <- Sys.time()

PARAM$ejecucion$duracion_segundos <- as.numeric(
  difftime(
    P12_FIN,
    P12_INICIO,
    units = "secs"
  )
)

PARAM$ejecucion$duracion_minutos <- round(
  PARAM$ejecucion$duracion_segundos / 60,
  2
)
```

y finalmente:

```
write_yaml(
  PARAM,
  file = "PARAM.yml"
)
```

Por lo tanto `PARAM.yml` es la fuente local donde quedan guardados:

```
escenario
semilla
training_pct
AUC
hiperparámetros ganadores
niter
registros antes/después
porcentaje real conservado
tiempo de ejecución
```

4 2.7 Qué hace cada script Python

Una vez terminados los cinco escenarios de una semilla, usamos:

```
p12_00_PROCESAR_SEMILLA_COMPLETA.py
```

Ese script es solamente un **orquestador**. No entrena modelos y no modifica Kaggle.

Ejecuta en orden:

```
p12_01_descargar_submissions.py
    |
    v
p12_02_consolidar_kaggle.py
    |
    v
p12_03_analizar_cortes.py
    |
    v
p12_04_integrar_resultados.py
```

4.1 Paso 1 - p12_01_descargar_submissions.py

Consulta Kaggle en modo lectura:

```
kaggle competitions submissions
```

y descarga los resultados que Kaggle ya calculó.

Genera:

```
90_RESULTADOS/  
  kaggle_submissions_completo.csv  
  
90_RESULTADOS/SEMx_SEED/  
  kaggle_SEMx_SEED_RAW.csv
```

El RAW contiene las submissions encontradas para esa semilla, incluyendo el publicScore .

4.2 Paso 2 - p12_02_consolidar_kaggle.py

Toma el RAW .

Si un mismo archivo fue enviado más de una vez, conserva la submission **más reciente**.

Después exige:

```
5 escenarios × 11 cortes = 55 submissions
```

Si están las 55, genera:

```
kaggle_SEMx_SEED_FINAL.csv
```

Si falta alguna, no genera el FINAL y detiene el procesamiento.

4.3 Paso 3 - p12_03_analizar_cortes.py

Toma:

```
kaggle_SEMx_SEED_FINAL.csv
```

y trabaja con:

```
fileName  
publicScore
```

Del nombre obtiene:

```
escenario  
semilla  
corte
```

y de publicScore obtiene la ganancia.

Genera dos archivos fundamentales:

```
resultados_cortes_SEMx_SEED.csv
resumen_maximos_SEMx_SEED.csv
```

resultados_cortes contiene:

```
5 escenarios × 11 cortes = 55 filas
```

resumen_maximos reduce esas 55 filas a:

```
5 escenarios = 5 filas
```

con:

```
corte_maximo
ganancia_maxima
cantidad_empates
```

Además genera una curva por escenario dentro de:

```
91_ANALISIS/SEMx_SEED/
```

4.4 Paso 4 - p12_04_integrar_resultados.py

Este es el script que **junta las dos ramas de información**.

Por un lado toma:

```
resumen_maximos_SEMx_SEED.csv
```

que contiene la información proveniente de Kaggle:

```
ganancia máxima
corte ganador
empates
```

Por otro lado abre los cinco:

```
PARAM.yml
```

de esa semilla y recupera:

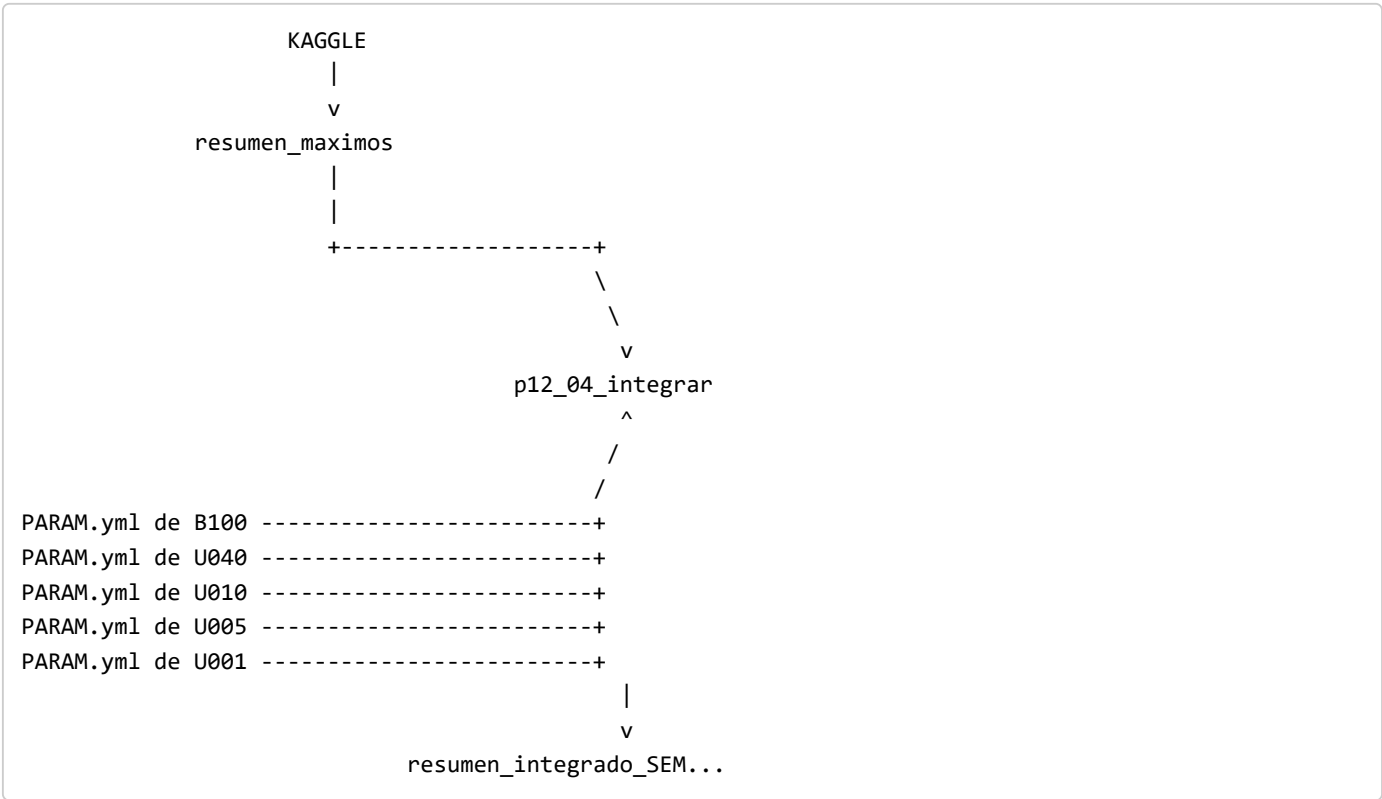
```
AUC
num_leaves
min_data_in_leaf
num_iterations
training_antes
training_despues
porcentaje_real_conservado
duracion_minutos
```

Finalmente genera:

```
resumen_integrado_SEMx_SEED.csv
```

Ese archivo es nuestra **fuentes principal para el análisis global**.

La lógica puede visualizarse así:



5 2.8 Entonces, ¿quién genera los archivos de 00_GLOBAL ?

Esta distinción es fundamental.

Los siguientes archivos **NO los genera el notebook**:

```
estado_semillas_P12.csv
tabla_maestra_P12.csv
resumen_por_escenario_P12.csv
comparacion_vs_baseline_P12.csv
detalle_cortes_todas_semillas.csv
```

Tampoco los generan directamente los scripts Python anteriores.

Los genera **este mismo archivo**:

```
P12_informe_control.Rmd
```

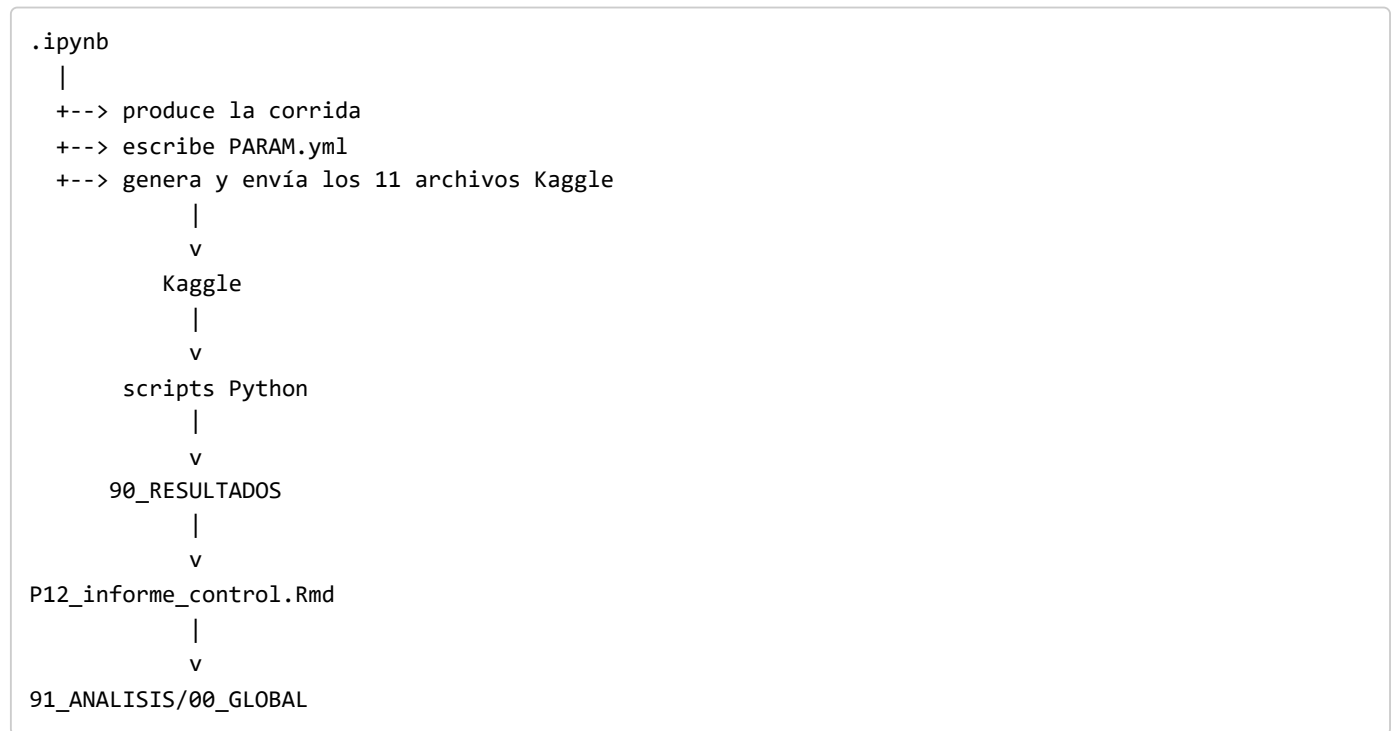
a partir de lo que ya existe en 90_RESULTADOS .

La relación exacta es:

Archivo de 91_ANALISIS/00_GLOBAL	Quién lo crea	Qué fuente usa
estado_semillas_P12.csv	este Rmd	existencia de archivos de 90_RESULTADOS y PARAMS_CORRIDAS
tabla_maestra_P12.csv	este Rmd	todos los resumen_integrado_SEM...csv completos
resumen_por_escenario_P12.csv	este Rmd	tabla_maestra_P12

Archivo de	91_ANALISIS/00_GLOBAL	Quién lo crea	Qué fuente usa
comparacion_vs_baseline_P12.csv	este Rmd		tabla_maestra_P12 , emparejando por semilla
detalle_cortes_todas_semillas.csv	este Rmd		todos los resultados_cortes_SEM...csv completos
gráficos globales .png	este Rmd		tabla_maestra_P12

Por lo tanto:



Esta es la trazabilidad que debemos conservar para saber siempre **de dónde salió cada resultado sin tener que reconstruir todo mentalmente**.

6 3. Estado automático de las corridas

Estado actual del experimento P12

SEM	semilla	ejecutado_por	modelos_con_PARAM	Kaggle_FINAL	cortes	maximos	integrado	estado
SEM1	198427	Victor	5	TRUE	TRUE	TRUE	TRUE	COMPLETA
SEM2	955939	Victor	5	TRUE	TRUE	TRUE	TRUE	COMPLETA
SEM3	602257	Victor	5	TRUE	TRUE	TRUE	TRUE	COMPLETA
SEM4	191099	Victor	5	TRUE	TRUE	TRUE	TRUE	COMPLETA
SEM5	895651	Victor	5	TRUE	TRUE	TRUE	TRUE	COMPLETA

Este cuadro es nuestro primer control: permite ver rápidamente qué semillas ya están listas para entrar en el análisis y cuáles todavía están incompletas.

7 4. Construcción automática de la tabla central

En este Knit se están analizando **5 semillas completas**, equivalentes a **25 combinaciones semilla + escenario**.

Tabla central del P12

SEM	Semilla	Escenario	Registros	% real conservado	AUC	Ganancia máxima	Corte	Tiempo (min)
1	198427	100%	179449	100.00	0.953599	19.238	850	14.03
1	198427	40%	72585	40.45	0.952189	19.404	850	11.34
1	198427	10%	19447	10.84	0.950708	19.238	850	8.85
1	198427	5%	10524	5.86	0.949474	19.071	1100 1250	8.96
1	198427	1%	3495	1.95	0.941342	18.155	900	7.13
2	955939	100%	179449	100.00	0.954227	21.153	800	15.03
2	955939	40%	73247	40.82	0.950986	20.654	950	10.42
2	955939	10%	19548	10.89	0.949491	21.070	800	8.56
2	955939	5%	10678	5.95	0.949453	20.570	950	7.50
2	955939	1%	3505	1.95	0.942084	20.404	950	6.93
3	602257	100%	179449	100.00	0.952214	22.403	850	13.85
3	602257	40%	72917	40.63	0.951866	22.403	850	11.28
3	602257	10%	19355	10.79	0.950998	22.403	850	8.74
3	602257	5%	10463	5.83	0.949090	21.487	900	8.25
3	602257	1%	3460	1.93	0.944867	21.403	1300	7.74
4	191099	100%	179449	100.00	0.952878	19.821	950	13.86
4	191099	40%	72876	40.61	0.952596	22.236	1250	12.00
4	191099	10%	19667	10.96	0.949655	20.820	1050	8.31
4	191099	5%	10696	5.96	0.947067	21.070	800	7.68
4	191099	1%	3483	1.94	0.938518	20.487	1050	7.08
5	895651	100%	179449	100.00	0.952001	17.572	1100	16.34
5	895651	40%	72782	40.56	0.953598	18.988	850	10.86
5	895651	10%	19547	10.89	0.951732	19.821	800 1250	8.67
5	895651	5%	10623	5.92	0.949156	17.572	800 1100	8.04
5	895651	1%	3394	1.89	0.943577	23.902	1200	7.60

Esta tabla es el corazón de nuestro informe. Cada fila representa una réplica del experimento bajo un determinado nivel de undersampling.

8 5. Resumen por escenario

Resumen descriptivo por nivel de undersampling

Escenario	N	Ganancia media	Ganancia mediana	SD ganancia	AUC medio	Registros medios	% real medio	Tiempo medio (min)
100%	5	20.037	19.821	1.844	0.952984	179449	100.00	14.62

Escenario	N	Ganancia media	Ganancia mediana	SD ganancia	AUC medio	Registros medios	% real medio	Tiempo medio (min)
40%	5	20.737	20.654	1.570	0.952247	72881	40.61	11.18
10%	5	20.670	20.820	1.220	0.950517	19513	10.87	8.63
5%	5	19.954	20.570	1.615	0.948848	10597	5.91	8.09
1%	5	20.870	20.487	2.074	0.942078	3467	1.93	7.30

Este cuadro permite empezar a responder nuestra pregunta principal sin mirar corrida por corrida: qué ocurre con la ganancia, el AUC, la cantidad de registros y el tiempo a medida que reducimos CONTINUA.

9 6. Comparación de cada escenario contra su propio Baseline

Para evitar comparar semillas distintas entre sí, cada escenario se compara con el **B100 de la misma semilla**.

Cambio respecto del Baseline de la misma semilla

SEM	Semilla	Escenario	Δ Ganancia vs 100%	Δ AUC vs 100%	Reducción registros %	Ahorro tiempo %
2	1	198427 40%	0.166	-0.001410	59.55	19.17
3	1	198427 10%	0.000	-0.002891	89.16	36.92
4	1	198427 5%	-0.167	-0.004125	94.14	36.14
5	1	198427 1%	-1.083	-0.012257	98.05	49.18
7	2	955939 40%	-0.499	-0.003240	59.18	30.67
8	2	955939 10%	-0.083	-0.004736	89.11	43.05
9	2	955939 5%	-0.583	-0.004774	94.05	50.10
10	2	955939 1%	-0.749	-0.012142	98.05	53.89
12	3	602257 40%	0.000	-0.000347	59.37	18.56
13	3	602257 10%	0.000	-0.001216	89.21	36.90
14	3	602257 5%	-0.916	-0.003124	94.17	40.43
15	3	602257 1%	-1.000	-0.007347	98.07	44.12
17	4	191099 40%	2.415	-0.000282	59.39	13.42
18	4	191099 10%	0.999	-0.003222	89.04	40.04
19	4	191099 5%	1.249	-0.005811	94.04	44.59
20	4	191099 1%	0.666	-0.014360	98.06	48.92
22	5	895651 40%	1.416	0.001597	59.44	33.54
23	5	895651 10%	2.249	-0.000269	89.11	46.94
24	5	895651 5%	0.000	-0.002845	94.08	50.80
25	5	895651 1%	6.330	-0.008424	98.11	53.49

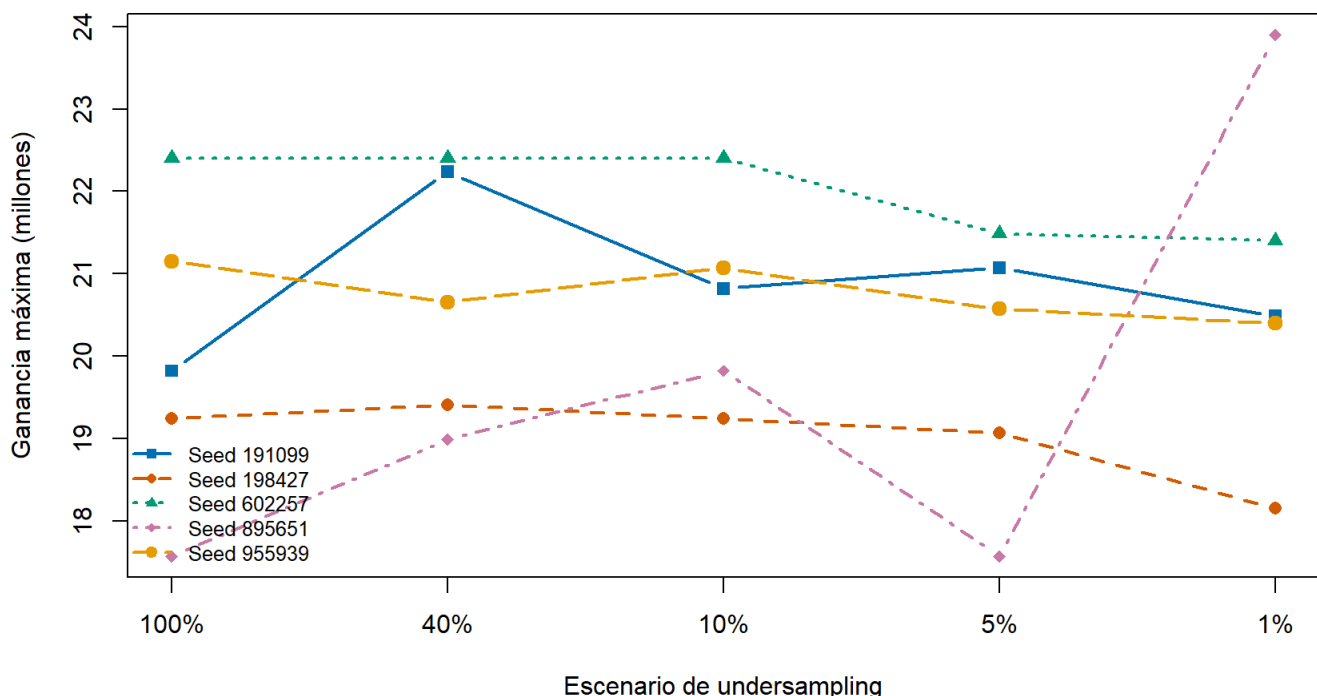
Esta es una de las tablas más importantes para interpretar el experimento:

- Δ Ganancia cercano a cero: la ganancia se mantiene próxima al Baseline.
- Δ AUC cercano a cero: el poder de ranking del modelo cambia poco.
- Reducción registros %: cuánto dataset evitamos procesar.
- Ahorro tiempo %: cuánto tiempo de cómputo se redujo.

10 7. Ganancia máxima

Convención visual de este informe: en los gráficos que comparan semillas, cada semilla conserva siempre el mismo color. En los gráficos de barras, cada escenario (100%, 40%, 10%, 5% y 1%) tiene un color propio. En el gráfico tiempo-ganancia se combinan color por semilla y símbolo por escenario. Esto permite seguir visualmente las réplicas sin depender solamente del tipo de línea.

Ganancia máxima por escenario y semilla



png
2

Qué mirar: no buscamos únicamente qué punto dio el score más alto. Nos interesa observar si la caída o estabilidad de la ganancia se repite entre semillas a medida que aumenta el undersampling.

10.0.1 Qué nos muestra este gráfico y qué indicios da

Este gráfico permite seguir **cómo cambia la ganancia máxima de cada semilla** cuando reducimos progresivamente los datos de entrenamiento.

Con las cinco semillas actuales aparecen varias señales:

- **40% y 10% mantienen ganancias muy competitivas respecto del Baseline (100%).**
- El escenario **10% muestra un comportamiento bastante estable entre semillas** y, al mismo tiempo, utiliza muchos menos registros.
- En **5%** empiezan a observarse más pérdidas frente a escenarios menos agresivos.

- El escenario **1% es claramente más inestable**: en varias semillas queda por debajo de escenarios menos reducidos, pero en SEM5 aparece un salto muy alto de ganancia (23.902) que empuja fuertemente la media.
- Por eso **no debemos elegir un escenario mirando solamente el punto máximo de una única semilla**.

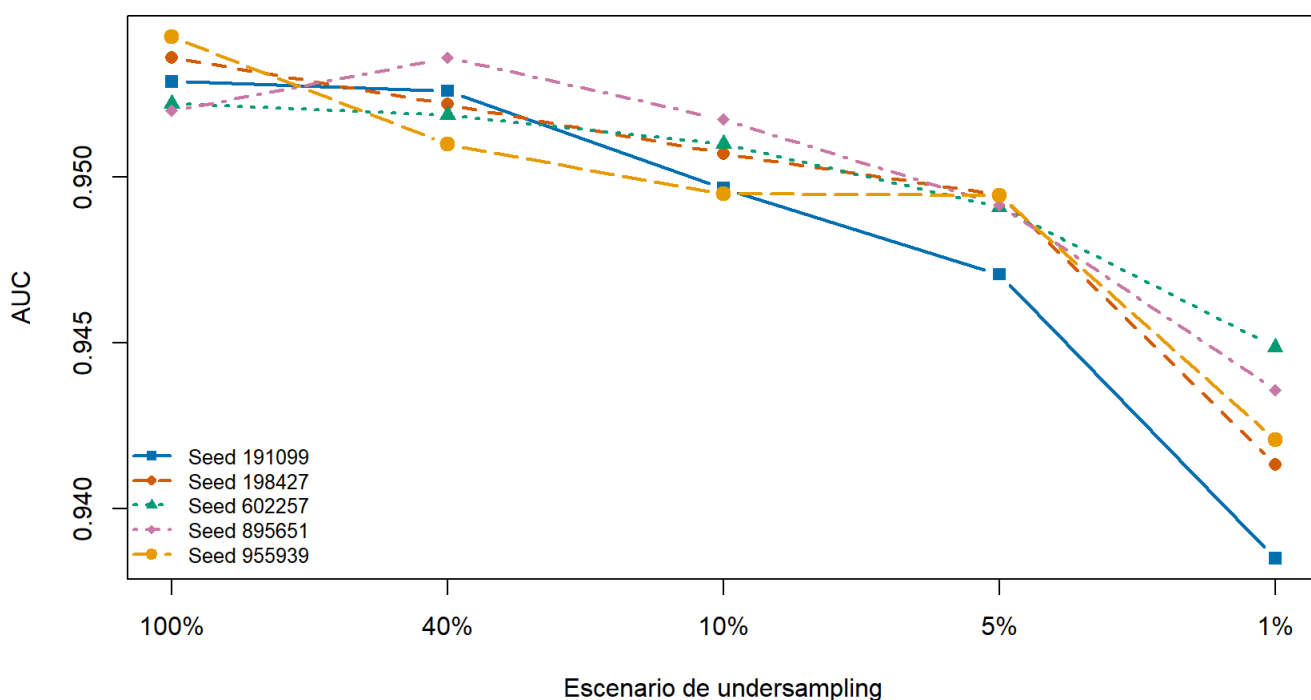
Una lectura preliminar del gráfico es:

El 10% aparece como una zona especialmente interesante de equilibrio entre reducción de datos y estabilidad de la ganancia.

Esta lectura todavía es provisoria porque, por ahora, corresponde a las cinco semillas de Victor. Cuando se incorporen las semillas de David, el mismo gráfico se actualizará automáticamente.

11 8. AUC

AUC por escenario y semilla



png

2

El AUC no reemplaza a la ganancia económica. Lo utilizamos como una segunda mirada para entender si la capacidad predictiva del modelo se deteriora cuando reducimos datos.

11.0.1 Qué nos muestra este gráfico y qué indicios da

A diferencia de la ganancia económica, el AUC presenta un patrón bastante más ordenado a medida que aumenta el undersampling.

Con las cinco semillas actuales, el AUC medio es aproximadamente:

100% = 0.952984
 40% = 0.952247
 10% = 0.950517
 5% = 0.948848
 1% = 0.942078

Los indicios principales son:

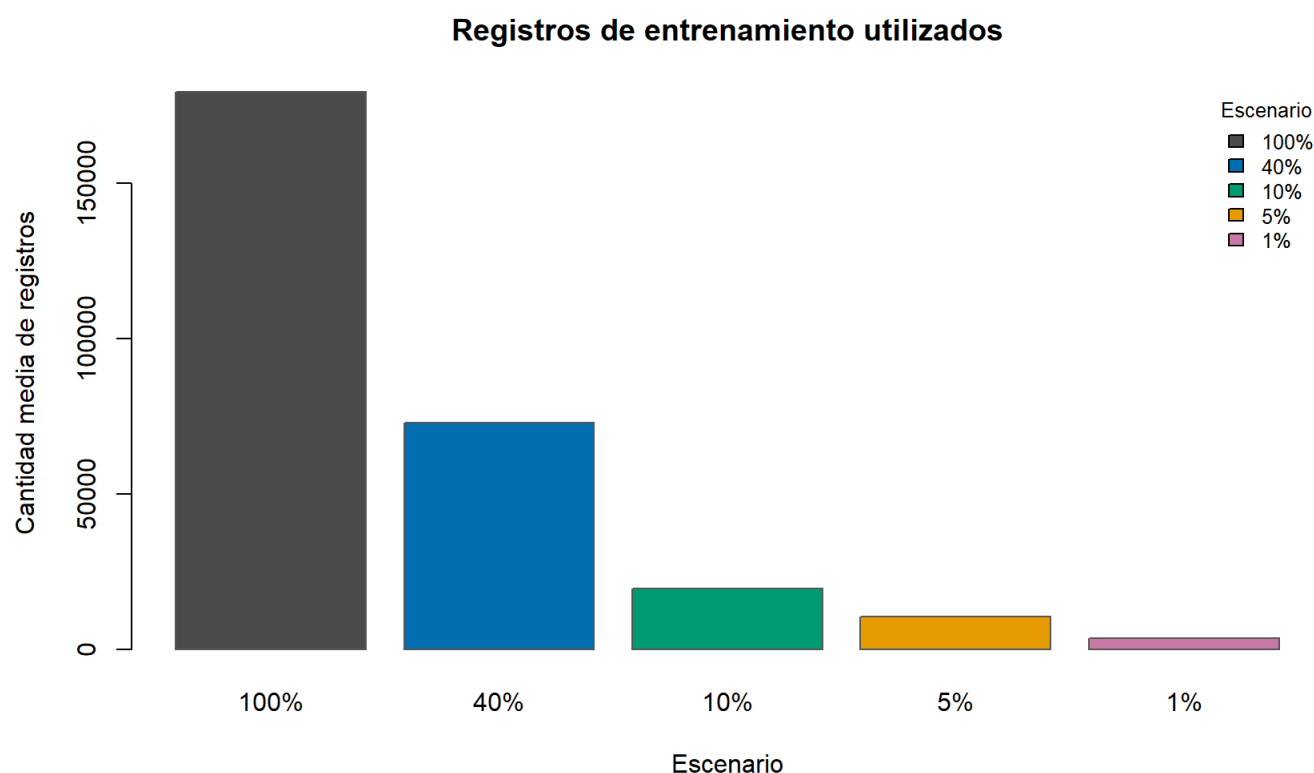
- **40% prácticamente conserva el poder predictivo del Baseline.**
- **10% todavía mantiene un AUC muy cercano al 100%**, pese a reducir fuertemente la cantidad de registros.
- En **5% comienza a verse una caída más clara.**
- En **1% el deterioro del AUC ya es bastante más marcado.**

Esto es importante porque ayuda a entender por qué una ganancia económica muy alta en una semilla aislada no alcanza para concluir que un escenario sea mejor. Por ejemplo, SEM5 obtiene una ganancia muy alta con 1%, pero el AUC muestra que ese escenario tiene menor capacidad global de ranking.

En resumen:

El AUC sugiere que el modelo soporta bastante bien la reducción hasta alrededor del 10%, mientras que por debajo de ese nivel comienzan a aparecer señales más claras de deterioro predictivo.

12 9. Registros utilizados



png

2

Acá visualizamos cuánto estamos reduciendo realmente el volumen de entrenamiento. Recordar: el porcentaje nominal se aplica a CONTINUA, no al total completo.

12.0.1 Qué nos muestra este gráfico y qué indicios da

Este gráfico cuantifica el beneficio directo del undersampling: **cuántos registros dejamos de procesar.**

Promedios actuales:

100% -> 179,449 registros
40% -> 72,881
10% -> 19,513
5% -> 10,597
1% -> 3,467

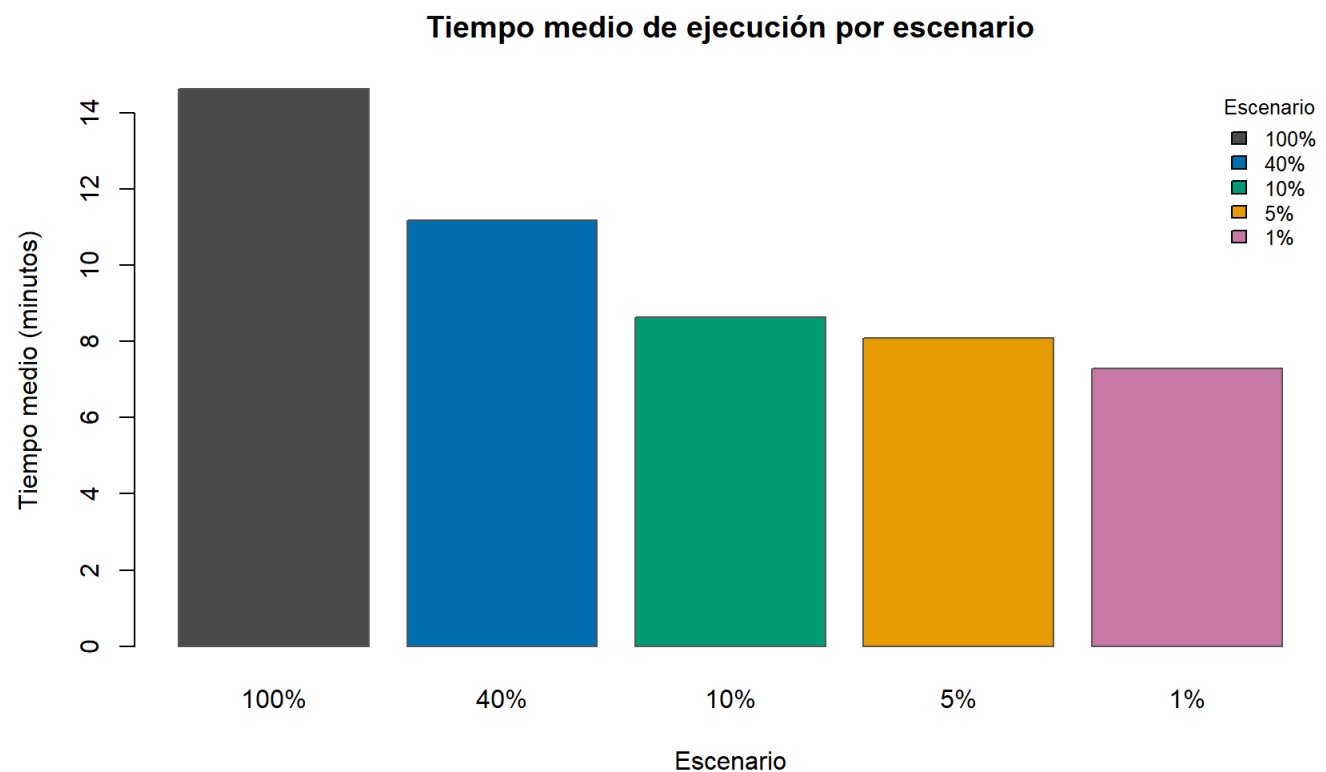
La reducción aproximada respecto del Baseline es:

40% -> 59% menos registros
10% -> 89% menos registros
5% -> 94% menos registros
1% -> 98% menos registros

El indicio más interesante vuelve a aparecer en **10%**: reduce cerca del 89% del volumen de entrenamiento, pero hasta ahora mantiene una ganancia y un AUC muy próximos a escenarios que usan muchos más datos.

Recordar que 10% significa aproximadamente 10% de CONTINUA ; BAJA+1 y BAJA+2 se conservan completas.

13 10. Tiempo de ejecución



```
## png  
## 2
```

Este gráfico permite cuantificar el beneficio computacional que acompaña al undersampling.

13.0.1 Qué nos muestra este gráfico y qué indicios da

El tiempo medio cae a medida que reducimos los registros:

100% -> 14.62 min
 40% -> 11.18 min
 10% -> 8.63 min
 5% -> 8.09 min
 1% -> 7.30 min

Respecto del Baseline, el escenario **10%** reduce aproximadamente un **41%** el tiempo medio.

También aparece una señal importante de **rendimientos decrecientes**:

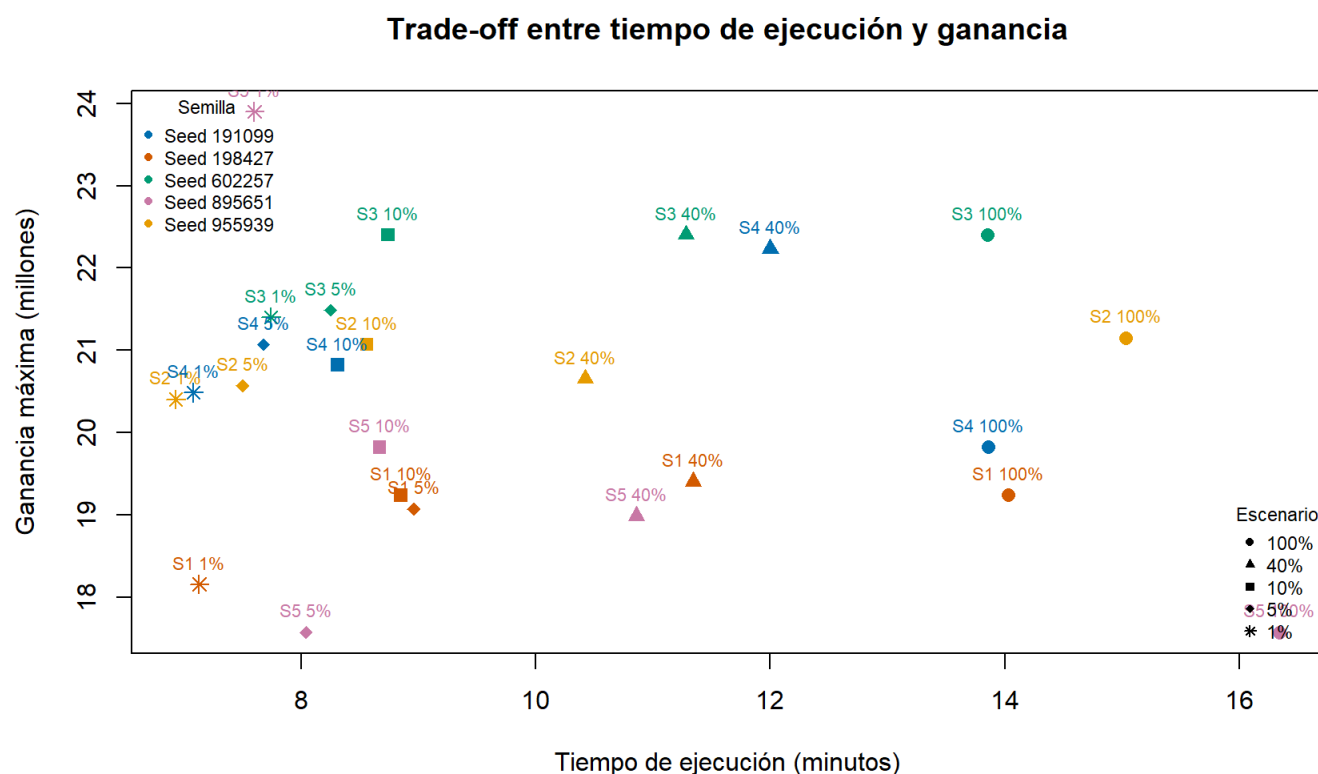
- pasar de 100% a 10% produce un ahorro considerable;
- bajar luego de 10% a 5% o 1% sigue reduciendo muchísimo la cantidad de registros;
- sin embargo, el ahorro adicional de tiempo es mucho menor.

Por eso no conviene perseguir el menor tiempo de ejecución de manera aislada. El análisis debe mirar en conjunto:

ganancia + AUC + registros + tiempo

El objetivo no es solamente entrenar más rápido, sino encontrar un nivel de undersampling que conserve un comportamiento suficientemente estable.

14 11. Relación tiempo - ganancia



png
 ## 2

Este gráfico **no implica que la ganancia sea una función del tiempo**. Simplemente coloca juntas dos métricas reales de cada corrida para observar el compromiso entre costo computacional y resultado económico.

14.0.1 Qué nos muestra este gráfico y qué indicios da

En este gráfico busquemos, de manera conceptual, puntos que estén **lo más arriba y a la izquierda posible**:

arriba -> mayor ganancia
izquierda -> menor tiempo de ejecución

No significa que exista una relación causal entre tiempo y ganancia; es una forma visual de estudiar el **trade-off** entre ambos objetivos.

Con las cinco semillas actuales:

- los escenarios **100%** consumen más tiempo;
- **40% y 10%** se desplazan hacia menores tiempos manteniendo ganancias competitivas;
- **10% aparece repetidamente en una zona atractiva** del gráfico;
- **1% tiene tiempos bajos, pero su ganancia es mucho más variable entre semillas.**

El indicio general es que:

10% podría estar cerca de una frontera eficiente: reduce fuertemente el costo computacional sin mostrar todavía un deterioro sistemático claro de la ganancia.

15 12. Hiperparámetros ganadores y niter

Configuración ganadora del Grid Search por semilla y escenario

SEM	Semilla	Escenario	num_leaves	min_data_in_leaf	niter
1	198427	100%	256	64	546
1	198427	40%	256	64	423
1	198427	10%	128	64	549
1	198427	5%	64	256	1815
1	198427	1%	64	64	449
2	955939	100%	256	64	420
2	955939	40%	256	64	269
2	955939	10%	256	64	451
2	955939	5%	256	64	289
2	955939	1%	64	64	368
3	602257	100%	256	256	456
3	602257	40%	256	64	595
3	602257	10%	128	64	572
3	602257	5%	64	64	778
3	602257	1%	64	64	1146
4	191099	100%	128	512	611
4	191099	40%	128	64	1124
4	191099	10%	128	64	671
4	191099	5%	64	64	558
4	191099	1%	64	64	834

SEM	Semilla	Escenario	num_leaves	min_data_in_leaf	niter
5	895651	100%	256	64	223
5	895651	40%	256	64	557
5	895651	10%	64	64	701
5	895651	5%	128	64	386
5	895651	1%	64	64	1128

Estos valores son útiles para comprender el comportamiento del entrenamiento. **No significa que nosotros hayamos cambiado manualmente los hiperparámetros entre escenarios:** el mismo procedimiento de búsqueda se vuelve a ejecutar y puede seleccionar un ganador diferente.

`niter` es la mejor iteración de boosting determinada durante el entrenamiento; **no es el corte de Kaggle**.

16 13. Cortes de Kaggle

Los cortes evaluados son 800, 850, ..., 1300. Para el análisis principal conservamos la **ganancia máxima obtenida dentro de esos 11 cortes**, pero mantenemos todo el detalle para poder revisar la curva cuando sea necesario.

Corte ganador por escenario y semilla

SEM	Semilla	Escenario	Ganancia máxima	Corte(s) máximo(s)	Cantidad de empates
1	198427	100%	19.238	850	1
1	198427	40%	19.404	850	1
1	198427	10%	19.238	850	1
1	198427	5%	19.071	1100 1250	2
1	198427	1%	18.155	900	1
2	955939	100%	21.153	800	1
2	955939	40%	20.654	950	1
2	955939	10%	21.070	800	1
2	955939	5%	20.570	950	1
2	955939	1%	20.404	950	1
3	602257	100%	22.403	850	1
3	602257	40%	22.403	850	1
3	602257	10%	22.403	850	1
3	602257	5%	21.487	900	1
3	602257	1%	21.403	1300	1
4	191099	100%	19.821	950	1
4	191099	40%	22.236	1250	1
4	191099	10%	20.820	1050	1
4	191099	5%	21.070	800	1
4	191099	1%	20.487	1050	1

SEM	Semilla	Escenario	Ganancia máxima	Corte(s) máximo(s)	Cantidad de empates
5	895651	100%	17.572	1100	1
5	895651	40%	18.988	850	1
5	895651	10%	19.821	800 1250	2
5	895651	5%	17.572	800 1100	2
5	895651	1%	23.902	1200	1

17 14. Comparación de escenarios todos contra todos

Hasta ahora comparamos cada escenario contra su propio Baseline (100%).

Eso responde:

¿Qué cambia respecto del escenario sin undersampling?

Pero para estudiar correctamente el Problema #12 también necesitamos comparar **todos los escenarios entre sí**, porque buscamos detectar en qué punto la reducción de datos empieza a cambiar de manera sistemática el comportamiento del modelo.

Con cinco escenarios existen exactamente diez comparaciones:

100% vs 40%
 100% vs 10%
 100% vs 5%
 100% vs 1%

 40% vs 10%
 40% vs 5%
 40% vs 1%

 10% vs 5%
 10% vs 1%

 5% vs 1%

La comparación es **pareada por semilla**.

Por ejemplo:

40% vs 10%

 SEM1: ganancia 40% vs ganancia 10%
 SEM2: ganancia 40% vs ganancia 10%
 SEM3: ganancia 40% vs ganancia 10%
 SEM4: ganancia 40% vs ganancia 10%
 SEM5: ganancia 40% vs ganancia 10%

Nunca se compara una semilla con otra.

17.1 14.1 Construcción automática de las diez comparaciones

17.2 14.2 Resumen descriptivo de las diez comparaciones

En la tabla siguiente:

- A gana = cuántas semillas tuvieron mayor ganancia en el escenario A;
- Empates = cuántas semillas dieron exactamente la misma ganancia máxima;
- B gana = cuántas semillas favorecieron al escenario B;
- Delta media = promedio de Ganancia A - Ganancia B ;
- Delta mediana = mediana de Ganancia A - Ganancia B .

Por lo tanto:

Delta > 0 -> favorece A
Delta = 0 -> comportamiento equivalente en la medida observada
Delta < 0 -> favorece B

Comparación descriptiva de ganancias: todos los escenarios contra todos

Escenario A	Escenario B	N pares	A gana	Empates	B gana	Delta media	Delta mediana
100%	40%	5	1	1	3	-0.700	-0.166
100%	10%	5	1	2	2	-0.633	0.000
100%	5%	5	3	1	1	0.083	0.167
100%	1%	5	3	0	2	-0.833	0.749
40%	10%	5	2	1	2	0.067	0.000
40%	5%	5	5	0	0	0.783	0.916
40%	1%	5	4	0	1	-0.133	1.000
10%	5%	5	4	0	1	0.716	0.500
10%	1%	5	4	0	1	-0.200	0.666
5%	1%	5	4	0	1	-0.916	0.166

Esta tabla todavía **no habla de significancia estadística**. Primero nos permite ver el patrón de dirección de las diferencias entre escenarios.

17.3 14.3 Wilcoxon pareado preliminar - todos contra todos

Como las mismas semillas se utilizan en los cinco escenarios, corresponde una comparación **pareada**.

Para cada una de las diez parejas se aplica:

```
wilcox.test(
  ganancia_A,
  ganancia_B,
  paired = TRUE,
  exact = FALSE
)
```

Se utiliza `exact = FALSE` porque en nuestros datos pueden existir empates o diferencias iguales a cero.

IMPORTANTE: con las cinco semillas actuales este Wilcoxon se considera **PRELIMINAR**.

Cuando se incorporen las semillas de David, el mismo bloque se recalculará automáticamente con el nuevo número de pares.

17.4 14.4 Tabla de Wilcoxon preliminar

Wilcoxon pareado preliminar: todos contra todos

Escenario A	Escenario B	N pares	N diferencias no cero	Mediana delta	W	p-value	p-value Holm
100%	40%	5	4	-0.166	2	0.2733	1.0000
100%	10%	5	3	0.000	1	0.2850	1.0000
100%	5%	5	4	0.167	6	0.7150	1.0000
100%	1%	5	5	0.749	9	0.6858	1.0000
40%	10%	5	4	0.000	5	1.0000	1.0000
40%	5%	5	5	0.916	15	0.0431	0.4311
40%	1%	5	5	1.000	10	0.5002	1.0000
10%	5%	5	5	0.500	13	0.1380	1.0000
10%	1%	5	5	0.666	10	0.5002	1.0000
5%	1%	5	5	0.166	10	0.5002	1.0000

17.4.1 Cómo leer esta tabla

El p-value responde:

¿Las diferencias pareadas de ganancia observadas entre estos dos escenarios muestran evidencia de un cambio sistemático entre semillas?

17.4.2 Qué significa realmente el p-value

El p-value **NO** indica cuál escenario es mejor y tampoco significa que un modelo sea “más certero”.

En nuestro Wilcoxon, la hipótesis nula puede pensarse de forma simple como:

Entre el escenario A y el escenario B no existe una diferencia sistemática de ganancia entre semillas.

Entonces:

p-value pequeño
|
v
más evidencia contra la hipótesis nula
|
v
hay indicios de que A y B se comportan de manera diferente

p-value grande
|
v
no hay evidencia suficiente para afirmar
que A y B tengan una diferencia sistemática

Por lo tanto:

Un p-value más cercano a cero NO significa que esa fila sea “la mejor”.

Para saber hacia qué escenario apunta la diferencia también debemos mirar:

A gana
B gana
Empates
Mediana delta

La dirección se interpreta así:

Mediana delta > 0 -> favorece A
Mediana delta $= 0$ -> no aparece una dirección central clara
Mediana delta < 0 -> favorece B

17.4.3 Ejemplo actual: 40% vs 5%

En nuestras cinco semillas:

A = 40%
B = 5%

A gana = 5
Empates = 0
B gana = 0
Mediana delta = +0.916
p-value = 0.0431
p-value Holm = 0.4311

Descriptivamente el patrón es muy fuerte:

40% obtuvo mayor ganancia que 5% en las cinco semillas.

El Wilcoxon sin corrección produce:

p = 0.0431

que es menor que el umbral tradicional de 0.05 .

Si estuviéramos realizando **una única comparación**, eso sería una señal estadísticamente significativa.

17.4.4 ¿Qué es el p-value Holm?

En este experimento no hacemos una sola prueba: realizamos **10 comparaciones todos contra todos**.

Cuanto más pruebas hacemos, mayor es la posibilidad de encontrar algún p-value pequeño simplemente por azar.

La corrección de **Holm** hace el criterio más exigente para controlar este problema de comparaciones múltiples.

Conceptualmente:

```
1 comparación
  |
  v
menor riesgo de encontrar algo por azar

10 comparaciones
  |
  v
mayor riesgo de algún falso positivo

corrección Holm
  |
  v
criterio más exigente
```

En el caso 40% vs 5%:

```
p-value original = 0.0431
p-value Holm     = 0.4311
```

Después de considerar que hicimos diez comparaciones:

```
0.4311 > 0.05
```

por lo tanto **todavía no podemos declarar una diferencia estadísticamente significativa después de la corrección múltiple**.

Esto NO borra la señal descriptiva:

```
40% gana 5 de 5
Mediana delta = +0.916
```

Simplemente indica que, con cinco semillas y diez pruebas simultáneas, la evidencia todavía no es suficientemente fuerte para considerarla definitiva.

17.4.5 Ejemplo actual: 40% vs 10%

Tenemos:

A gana = 2
Empates = 1
B gana = 2
Mediana delta = 0.000
p-value = 1.000
p-value Holm = 1.000

No aparece una dirección consistente:

2 semillas favorecen 40%
2 semillas favorecen 10%
1 semilla empata

La lectura correcta NO es:

“10% es perfecto”.

La lectura es:

Con estas cinco semillas no vemos evidencia de una diferencia sistemática de ganancia entre 40% y 10%.

Y eso puede ser muy importante para P12 porque:

40% -> ~72,881 registros
10% -> ~19,513 registros

Si ambos mantienen una ganancia similar, el 10% consigue prácticamente el mismo resultado económico con muchísimo menos volumen de entrenamiento.

17.4.6 Ejemplo actual: 100% vs 10%

La comparación da aproximadamente:

A gana = 1
Empates = 2
B gana = 2
Mediana delta = 0
p-value = 0.285

Tampoco aparece evidencia fuerte de una diferencia sistemática de ganancia entre 100% y 10%.

Pero el 10% reduce aproximadamente:

89% de los registros
41% del tiempo medio

Por eso, en este experimento, un p-value grande puede ser compatible con algo que justamente nos interesa demostrar:

podemos reducir mucho los datos sin detectar una pérdida sistemática de ganancia.

17.4.7 Por qué 1% necesita cuidado especial

El escenario 1% presenta una variabilidad mucho mayor.

Por ejemplo, en 100% vs 1%:

100% gana = 3 semillas
1% gana = 2 semillas

pero SEM5 obtiene una ganancia excepcionalmente alta con 1% (23.902).

Eso provoca que:

Delta media -> pueda quedar muy influida por SEM5
Delta mediana -> represente mejor la dirección central de las cinco semillas

Al mismo tiempo, el AUC del 1% es claramente menor.

Esto refuerza la idea de que **no debemos decidir usando solamente la ganancia máxima ni solamente el p-value.**

17.4.8 Orden recomendado para interpretar cada comparación

Para nuestro P12 conviene leer cada fila en este orden:

1. A gana / B gana / Empates
2. Mediana delta
3. Magnitud de las diferencias
4. p-value
5. p-value Holm
6. AUC
7. Registros utilizados
8. Tiempo de ejecución

La pregunta del experimento no es:

“¿Qué fila tiene el p-value más bajo?”

La pregunta es:

¿Hasta cuánto podemos reducir los datos sin deteriorar de manera relevante y estable el modelo?

17.4.9 Indicio global actual con cinco semillas

La lectura provisional es:

Comparación	Indicio actual
100% vs 40%	No aparece una diferencia sistemática clara
100% vs 10%	No aparece una diferencia sistemática clara
40% vs 10%	Comportamiento de ganancia muy similar
40% vs 5%	40% gana en las 5 semillas: señal fuerte de cambio
10% vs 5%	10% gana 4 de 5: empieza a aparecer deterioro

Comparación	Indicio actual
10% vs 1%	10% gana 4 de 5, pero 1% es muy variable
5% vs 1%	Comparación también afectada por la fuerte variabilidad del 1%

Esto empieza a sugerir:

```

100%
|
v
40%  -> estable
|
v
10%  -> todavía estable y mucho más económico
|
v
-----
posible zona de quiebre
-----
|
v
5%   -> empieza a deteriorarse
|
v
1%   -> comportamiento más inestable

```

Esta es una **interpretación preliminar con cinco semillas**. Cuando se incorporen las semillas de David, el mismo Rmd recalculará automáticamente todas las tablas y tests con el nuevo número de réplicas.

Pero con las cinco semillas actuales debemos ser prudentes:

- el tamaño muestral todavía es pequeño;
- pueden existir diferencias iguales a cero;
- el objetivo actual es detectar patrones y preparar el análisis definitivo;
- las semillas adicionales de David aumentarán el número de pares;
- el resultado final no se decidirá únicamente por el *p-value*, sino también por magnitud de diferencias, dispersión, AUC y ahorro de recursos.

El *p-value* Holm se conserva como información adicional porque estamos realizando diez comparaciones simultáneamente. No reemplaza el criterio que finalmente indique el profesor para el trabajo.

17.5 14.5 Qué archivos nuevos deja esta sección

Después de cada Knit se actualizan automáticamente:

```

91_ANALISIS/00_GLOBAL/
├─ detalle_comparaciones_todos_contra_todos_P12.csv
├─ comparaciones_todos_contra_todos_P12.csv
└─ wilcoxon_todos_contra_todos_P12.csv

```

Por lo tanto, cuando incorporemos nuevas semillas no habrá que rehacer las comparaciones manualmente:

```

nuevos resumen_integrado
  |
  v
tabla_maestra
  |
  v
Knit
  |
  +--> comparaciones todos contra todos
  |
  +--> Wilcoxon pareado
  |
  v
tablas actualizadas automáticamente

```

18 15. ¿Estamos listos para la comparación estadística?

Control previo a la inferencia estadística

control	resultado
Semillas completas disponibles	5
Los 5 escenarios están presentes	SI
Cada semilla aporta los mismos 5 escenarios	SI
Wilcoxon final ejecutado en este informe	NO - se deja para la etapa final

Por ahora este informe **no ejecuta automáticamente Wilcoxon**. La razón es deliberada: primero queremos completar las réplicas previstas y tener todas las ganancias consolidadas.

Cuando estén las semillas de Víctor y David que finalmente se decida utilizar, la comparación se hará **pareada por semilla**, porque los cinco escenarios fueron ejecutados sobre las mismas semillas.

19 16. Qué tenemos hoy y qué falta

```
## SEMILLAS COMPLETAS:
```

```
## SEM1, SEM2, SEM3, SEM4, SEM5
```

```
## SEMILLAS TODAVÍA NO COMPLETAS:
```

```
## Ninguna
```

Con cada actualización de 90_RESULTADOS basta con volver a hacer **Knit**. No hay que reconstruir tablas a mano.

20 17. Archivos automáticos que deja este informe

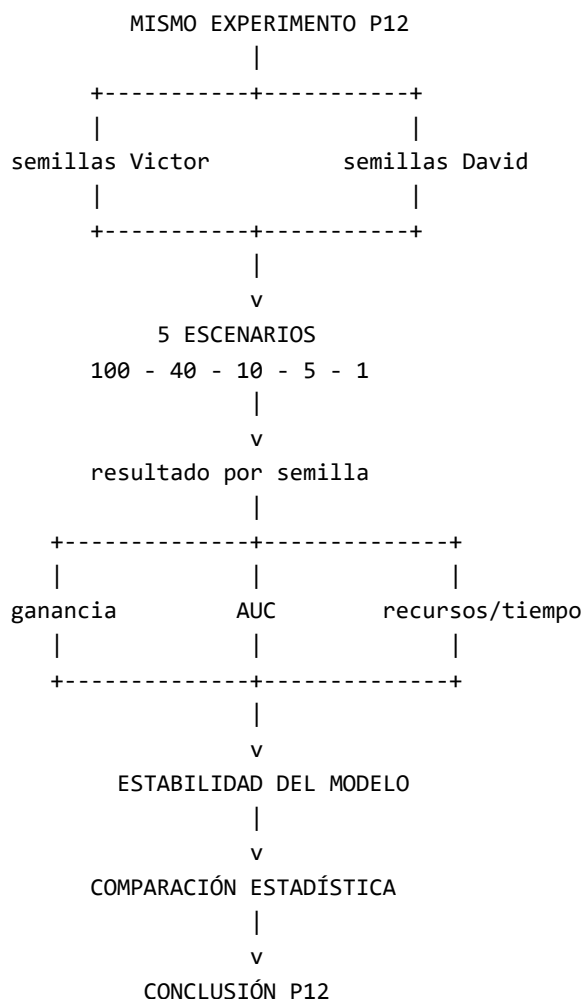
Cada ejecución actualiza:

```

91_ANALISIS/
├── 00_GLOBAL/
│   ├── estado_semillas_P12.csv
│   ├── tabla_maestra_P12.csv
│   ├── resumen_por_escenario_P12.csv
│   ├── comparacion_vs_baseline_P12.csv
│   ├── detalle_cortes_todas_semillas.csv
│   ├── detalle_comparaciones_todos_contra_todos_P12.csv
│   ├── comparaciones_todos_contra_todos_P12.csv
│   ├── wilcoxon_todos_contra_todos_P12.csv
│   └── GRAFICOS/
│       ├── 01_ganancia_por_semilla.png
│       ├── 02_auc_por_semilla.png
│       ├── 03_registros_promedio.png
│       ├── 04_tiempo_promedio.png
│       └── 05_tiempo_vs_ganancia.png

```

21 18. Mapa mental del P12



21.1 Idea fundamental

No buscamos elegir el escenario con el número más alto de una corrida aislada.

Buscamos determinar, mediante **varias semillas del mismo experimento**, hasta qué nivel puede reducirse la información de entrenamiento manteniendo un comportamiento suficientemente estable, y cuánto recurso computacional se ahorra a cambio.

Este R Markdown queda como nuestro **documento vivo de control**: los scripts producen los datos; el informe los ordena, los grafica y conserva la trazabilidad de cómo llegamos a cada conclusión.

21.2 Lectura provisional actual

Con las cinco semillas disponibles, el patrón que más interesa seguir es:

```
40% y 10%  
  |  
  v  
mantienen un comportamiento muy competitivo  
  |  
  v  
10% reduce muchísimo registros y tiempo  
  |  
  v  
5% empieza a mostrar señales de deterioro  
  |  
  v  
1% muestra mayor inestabilidad
```

La hipótesis de trabajo que surge por ahora es:

El modelo parece robusto frente al undersampling hasta alrededor del escenario 10%, donde todavía conserva ganancias comparables al Baseline y un AUC cercano, utilizando muchos menos registros y menor tiempo de ejecución. Por debajo de ese nivel comienzan a aparecer señales de deterioro e inestabilidad.

Esta frase es **provisoria** y debe volver a evaluarse cuando se incorporen las semillas de David y se complete el análisis estadístico final.